

Design Patterns: Behavioural & Creational

CCP6214

Object-oriented Analysis and Design

Portions of this file have content that were adapted from [1] Object-Oriented Software Engineering Using UML, Patterns, and Java, [2] OOSE Practical software development using UML and Java, [3] <http://kremer.cpsc.ucalgary.ca/patterns/index.html>, [4] <http://www.tutorialspoint.com/index.htm>, [5] <http://www.avajava.com/>, [6] <https://dzone.com>, [7] https://sourcemaking.com/design_patterns/structural_patterns and [8] <http://uosis.mif.vu.lt/~plukas/resources/DPBook/excerpts/patterns.htm>

Lecture outcomes

- At the end of this lecture, you should
 - learn about behavioural and creational design patterns and its uses
 - identify and distinguish various common design patterns
 - identify how to use DPs depending on scenarios

Recap

Creational	Structural	Behavioral
Builder Prototype Singleton Factory Method Abstract Factory	Adapter Bridge Composite Façade Decorator Flyweight Proxy	Iterator Observer Interpreter Template Method Chain of Responsibility Command Mediator Memento State Strategy Visitor

Design Pattern content recap

- A typical design pattern has
 - **Name** - A meaningful pattern identifier
 - **Intent** - What does the pattern do? What is its rationale?
 - **Motivation** (Problem Description) - A scenario that illustrates a design problem and how the class structures in the pattern solve the problem
 - **Applicability** - What are situations in which the pattern can be applied?
 - **Structure** (Solution) - Description of the pattern by means of standardized representation techniques (class diagrams, UML...)
 - **Consequences** - The results and trade-offs of applying the pattern.
 - **Sample Code/Examples**

Behavioural Design Patterns

- In OO software engineering, behavioral design patterns are patterns that identify common communication patterns between objects and develop design based on these communications
- This increases the flexibility by letting them talk to each other but still retaining the desired loose coupling between these objects
- Behavioral DPs are concerned with the assignment of responsibilities between objects, or, encapsulating behavior in an object and delegating requests to it
- Behavioural class DPs typically use inheritance to distribute behaviour between classes
- Behaviour object DPs use composition instead of inheritance to achieve the same.



COMMON DESIGN PATTERNS

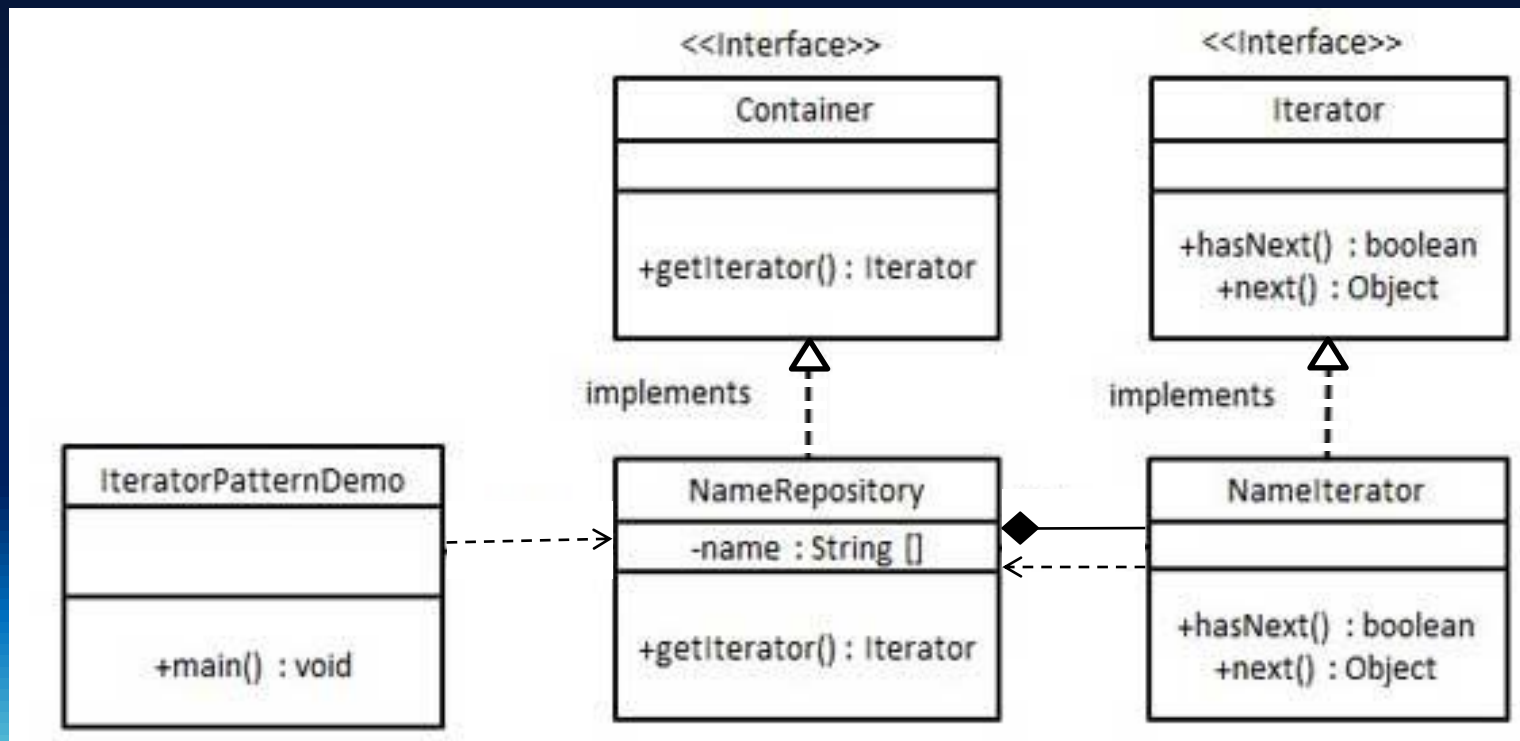
(Iterator- Behavioural)

Iterator DP scenario

- There are hundreds of free online radio channels, and you want to write a program that will allow users to navigate each channel easily using one interface
- The user does not need to know about the channel order but can navigate through the result list and can still manipulate each channel – play, stop, volume etc – by scanning through them one-by-one based on a request type (e.g. only classical)

- The iterator DP is most suitable for this because it allows the user to traverse/manipulate through a collection of objects without knowing details like the number of objects being retrieved
- The pattern hides the actual implementation of traversal through the collection and client programs just use iterator methods and access the objects as usual

- An iterator allows a client to access a contents of an aggregate data structure without knowing about the internal representation of the content
- Iterator DP is also sometimes called “Cursor”
- An example of the Iterator DP is shown below (other variations possible)



How is it implemented?

- Interfaces are created to represent an “Iterator” and a “Container” to store iterated objects

```
public interface Container {  
    public Iterator getIterator();  
}
```

```
public interface Iterator {  
    public boolean hasNext();  
    public Object next();  
}
```

- The “Container” would be used as object/datatype in the user program – it implements the “Iterator” object
- The “Iterator” has functionality to navigate all the objects – in the example above, it can retrieve the next available object with **next()** and test whether any objects are remaining with **hasNext()**

- To *implements* the “Container” for the radio channel example (assuming that each radio channel is represented by the datatype RdChannel)
- The abstract getIterator() method would have to return a new Iterator object which is the concrete realisation of the Iterator interface

```
public class RadioControl implements Container {  
    public RdChannel[] names = {ch1,ch2,ch3,ch4};  
  
    @Override  
    public Iterator getIterator() {  
        return new ChannelIterator();  
    }  
}
```



Note that this class hasn't been created yet in this code!

- To complete the Container class, the concrete Iterator class is added (note that it can also be created under a separate file)

```
public class RadioControl implements Container {  
    public RdChannel[] names = {ch1,ch2,ch3,ch4};  
    @Override  
    public Iterator getIterator() {  
        return new ChannelIterator();  
    }  
}
```

```
public class ChannelIterator implements Iterator {  
    int index;  
    public boolean hasNext(){  
        if (index<names.length) return true;  
        else return false;  
    }  
    public RdChannel next(){  
        if (names.hasNext())  
            return names[index++];  
        else  
            return null;  
    }  
}
```



```
public interface Iterator{  
    public boolean hasNext();  
    public Object next();  
}
```

- Whats remaining is to use the Container object in the user application code

```
public class MyRadioPlayer {  
  
    public static void main(String[] args) {  
        RadioControl rd = new RadioControl();  
  
        while(rd.hasNext()){  
            RdChannel currentCh = rd.Next();  
            // do something with currentCh object, e.g.  
            // System.out.println(currentCh);  
        }  
    }  
}
```

How is it implemented? Again?

- Java actually has built in interface object called Iterator that works with objects
- Has methods ready for next, hasNext and remove
- Iterator also works with List objects
- <http://docs.oracle.com/javase/7/docs/api/java/util/Iterator.html>

```
List<String> list = new ArrayList<String>();  
// add strings  
// list.add("sample 1");  
  
Iterator it = list.iterator();  
while(it.hasNext())  
{  
    String s = it.next();  
}
```

Iterator DP summary

Name	Iterator
Motivation	To provide a method to sequentially access elements of a collection type object without needing to know its underlying representation
Applicability	Used when • Need to access an aggregate object's contents without exposing its internal representation • Need to support multiple traversals of aggregate objects • Need to provide a uniform interface for traversing different aggregate structures
Solution	Define an iterator that fetches one element at a time Each iterator object keeps track of the position of the next element If there are several aggregate/iterator variations, it is best if the aggregate and iterator classes realize common interface types
Consequences	Iterator Pattern supports variations in the traversal of an aggregate. Every time you want to change the traversal algorithm, you just replace the iterator instance with a different one. Iterators simplify the Aggregate interface. More than one traversal can be pending on an aggregate. An iterator keeps track of its own traversal state. Therefore you can have more than one traversal in progress at once

For more information on Iterator

- <https://refactoring.guru/design-patterns/iterator>
- Google “Iterator Design Pattern”



COMMON DESIGN PATTERNS

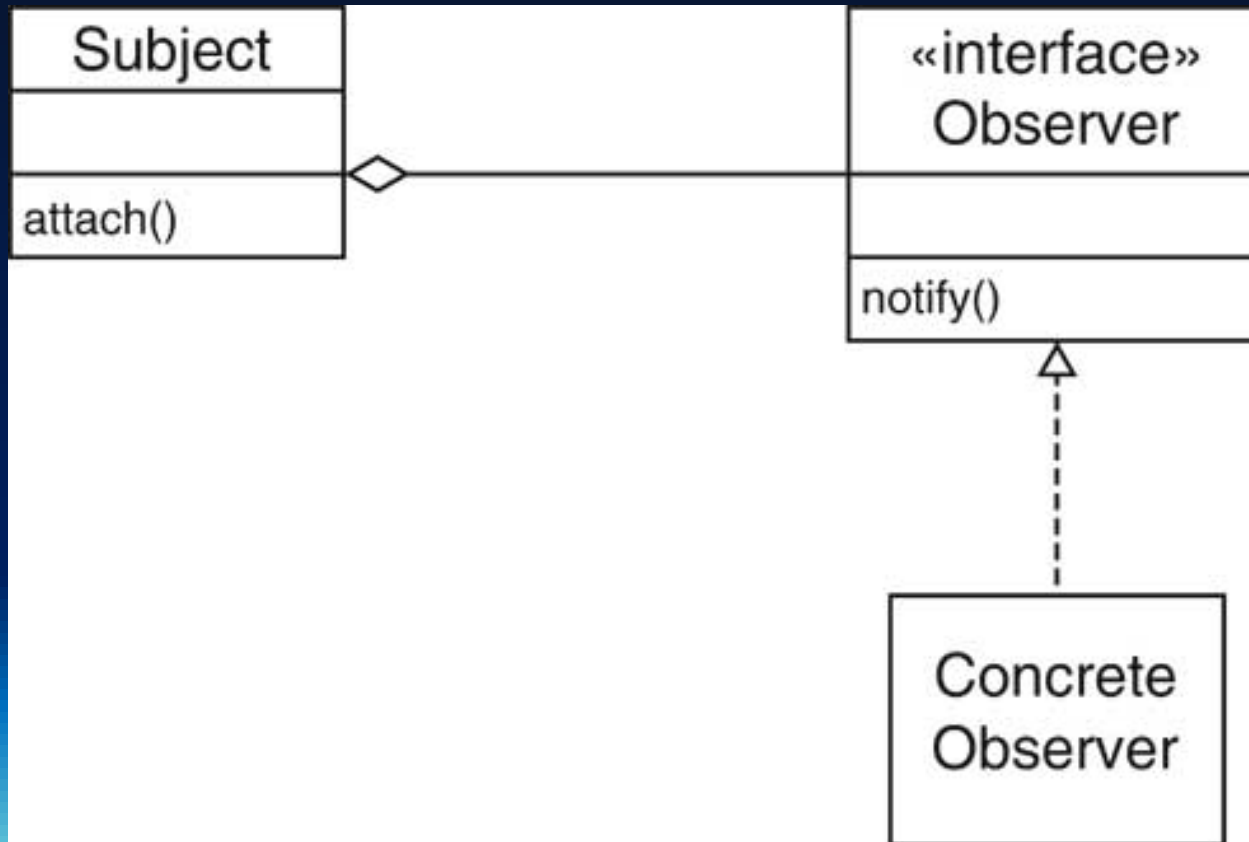
(Observer - Behavioural)

Observer DP scenario

- You are writing a MMORPG game that groups multiple players into “tribes” that fight each other
- Each time your character makes a “kill” it gains points – when points reach a certain level, your character unlocks certain abilities/features. The entire tribe will also know about your “upgrades”
- There are also other actions that can gain hidden achievements for your character

- Although it would be easy to code this into the “character” object, things might get messy fast – imagine multiple counters per player keeping track of kills, coins, jewels, etc that must be repeatedly checked – and will slow down performance.
- A better solution is to use the Observer DP – this is related to the model-view-control (MVC) DP.

- In the Observer DP, an object (referred to as subject) becomes the source of events – it has items that other objects (observers) can monitor on their own.
- Observer objects (1 or more) ‘listen’ to subjects and want to be notified when such an event occurs to trigger an action



How is it implemented?

- In the scenario, the player character is the subject for the observer to “observe”. The character object should:
 - keep a list of observers
 - be able to add/remove observers as and when necessary
 - be able to notify and/or get updates from observers
- An example of the subject interface could be as follows

```
public interface Subject {  
  
    public void register(Observer obj);  
    public void unregister(Observer obj);  
  
    public void notifyObservers();  
  
    public Object getUpdate();  
  
}
```

to add/remove
observers



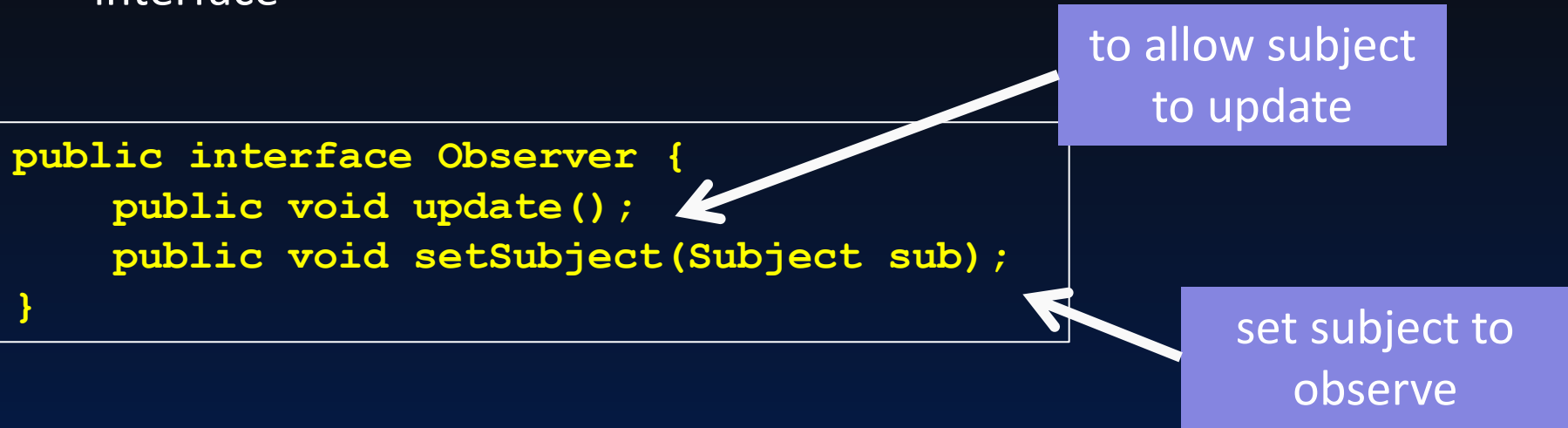
send update to
observers



- On the opposite end, the Observer can possibly have the following interface

```
public interface Observer {  
    public void update();  
    public void setSubject(Subject sub);  
}
```

to allow subject
to update



set subject to
observe

- Implementing the concrete class of the subject

```
public class Character implements Subject {
    int numOfKills;
    // other variables
    // other methods
    List<Observer> obList = new List<Observer>();
    public void register(Observer obj){
        obList.add(obj);
    }
    public void unregister(Observer obj) {
        obList.remove(obj);
    }
    public void notifyObservers(){
        // some condition should trigger this method
        for (Observers o:obList){
            o.update();
        }
    }
    public Object update(){ return numOfKills; }
}
```

- Implementing the concrete class of the observer

```
public class GAcHeivements implements Observer {
    private Subject monitoredItems;
    // other variables
    // other methods
    public void update() {
        int value = monitoredItems.getUpdate();
        if (value > 400)
            // unlock acheivement action here
        // etc ....
    }
    public void setSubject(Subject sub) {
        this.monitoredItems = sub;
    }
}
```

Observer DP summary

Name	Observer
Motivation	To provide a method to represent a one-to-many relationship between objects such that when one object is modified, dependent objects are notified automatically
Applicability	Use Observer when • an abstraction has two aspects, one dependent on the other. • a change to one object requires changing others, and you don't know how many objects need to be changed. • an object should be able to notify other objects without making assumptions about who these objects are.
Solution	Define an observer interface type. All concrete observers implement the observer interface. The subject maintains a collection of observers. The subject supplies methods for attaching and detaching observers. Whenever an event occurs, the subject notifies all observers.
Consequences	Potential disadvantage when successive/repeated updates to observers when there is a series of incremental changes to the data Broadcasted communication – notification sent from a subject does not specify receiver and will broadcast to all subscribed parties Unexpected update – observers do not know about each other so a small update may trigger a cascade of updates

For more information on Observer

- <https://www.geeksforgeeks.org/observer-pattern-set-1-introduction/>
- For more information, Google “Observer Design Pattern”

Interpreter - Behavioural

- The interpreter pattern is used to define the grammar for instructions that form part of a language or notation, whilst allowing the grammar to be easily extended.
- <https://www.codingdrills.com/tutorial/design-patterns-tutorial/interpreter-pattern>
- For more information, Google “Interpreter Design Pattern”

Template Method - Behavioural

- The Template Method pattern defines a skeleton of an algorithm in a method of a superclass (often abstract or base class) while allowing subclasses to provide specific implementations of some of the method steps. The key idea is to let subclasses determine certain steps of the algorithm without changing the algorithm's structure itself.
- Parts:
 - Define a High-Level Algorithm
 - Encapsulate Variable Steps
 - Override in Subclasses
 - Polymorphic Behavior
- <https://www.geeksforgeeks.org/template-method-design-pattern-in-java/>
- For more information, Google "Template Method Design Pattern"

Chain of Responsibility - Behavioural

- The chain of responsibility pattern is used to process varied requests, each of which may be dealt with by a different handler.
- Some similarity with Decorator and Composite.
 - Decorator:
 - Similarity: wrapping.
 - Difference: each in CoR may terminate the passing on.
 - Composite:
 - Similarity: may involve hierarchy.
 - Difference: Composite emphasises treating group uniformly, CoR more emphasises sequential/prioritized processing flow.
- <https://refactoring.guru/design-patterns/chain-of-responsibility>
- For more information, Google “Chain of Responsibility Design Pattern”

Command - Behavioural

- The command pattern is used to express a request, including the call to be made and all of its required parameters, in a command object. The command may then be executed immediately or held for later use.
- <https://refactoring.guru/design-patterns/command>
- For more information, Google “Command Design Pattern”

Mediator - Behavioural

- The mediator pattern is used to reduce coupling between classes that communicate with each other. Instead of classes communicating directly, and thus requiring knowledge of their implementation, the classes send messages via a mediator object.
- How Mediator is different from Façade and Adapter:

Pattern	Purpose	Focus
Mediator	Manage complex interactions between multiple objects to reduce dependencies and coupling	Centralizes control and communication
Facade	Simplify access to a complex subsystem by providing a high-level interface	Simplifies interaction with a system but doesn't manage internal communication
Adapter	Make incompatible interfaces compatible	Enables objects to work together by translating interfaces

- <https://daily.dev/blog/mediator-design-pattern-explained>
- For more information, Google “Mediator Design Pattern”

Memento - Behavioural

- The memento pattern is used to capture the current state of an object and store it in such a manner that it can be restored at a later time without breaking the rules of encapsulation.
- How is it different from Command?
 - **Memento**: restore an object to previous state & don't expose internal details.
 - **Command**: encapsulate actions as objects, no need full obj state.
 - Both can do undo, but **Memento**: *state-based undo*, while **Command**: *action-based undo*.

Memento - Behavioural

Aspect	Memento Pattern	Command Pattern
Primary Purpose	Capturing and restoring state	Encapsulating an action or request as an object
Undo Mechanism	Restores previous state snapshots	Reverses actions by performing complementary steps
Implementation	Stores the full state of an object at a point	Stores actions and data needed to reverse them
Best for	State restoration or undo to a specific checkpoint	Action-based undo or command queuing/logging
Examples	Saving/restoring text editor state, checkpoints	Undo/redo actions in GUI commands, transactions

- <https://www.geeksforgeeks.org/memento-design-pattern/>
- For more information, Google “Memento Design Pattern”

State - Behavioural

- The state pattern is used to alter the behaviour of an object as its internal state changes. The pattern allows the class for an object to apparently change at run-time.
- <https://www.geeksforgeeks.org/state-design-pattern/>
- For more information, Google “State Design Pattern”

Strategy - Behavioural

- The strategy pattern is used to create an interchangeable family of algorithms from which the required process is chosen at run-time.
- This may sound similar to State, so here's a comparison:

Aspect	State	Strategy
Main purpose	Allow an object to change its behavior based on its internal state	Allow an object to change its behavior based on its internal state
Context	Behavior changes due to an object's current state	Behavior changes due to a chosen strategy or algorithm
Self-awareness	States often know about and can switch to each other	Strategies are typically unaware of each other
Primary Use Case	Managing different states of an object (e.g., document states)	Choosing between different ways to achieve a specific goal
State Transition	States can transition to other states (often through the Context)	States can transition to other states (often through the Context)

- <https://refactoring.guru/design-patterns/strategy>
- For more information, Google "Strategy Design Pattern"

Visitor

- The visitor pattern is used to separate a relatively complex set of structured data classes from the functionality that may be performed upon the data that they hold.
- Different from iterator in that **Visitor** is about applying various operations to a structure's elements while **Iterator** is about providing a standard way to access elements in sequence. The two patterns can work together: you might use an iterator to traverse elements in a collection, and for each element, apply a visitor to perform an operation.
- <https://www.geeksforgeeks.org/visitor-design-pattern/>
- For more information, Google “Visitor Design Pattern”

Implementation differences in DPs

- Behavioural patterns are concerned with the assignment of responsibilities between objects, or, encapsulating behaviour in an object and delegating requests to it.
- Often behavioural DPs may make use of structural DPs (e.g. Iterator usually works with Composite)
- The Observer DP is basically a publish-subscribe pattern – subjects publish items that observers subscribe to (if interested)
- The Observer creates a one-to-many relationship between the subject and its observers.

Creational Design Patterns

- Creational patterns increase flexibility and reuse of existing code by providing various object creation mechanisms.
- According to https://sourcemaking.com/design_patterns, “these design patterns are all about class instantiation. This pattern can be further divided into class-creation patterns and object-creational patterns. While class-creation patterns use inheritance effectively in the instantiation process, object-creation patterns use delegation effectively to get the job done.”





COMMON DESIGN PATTERNS

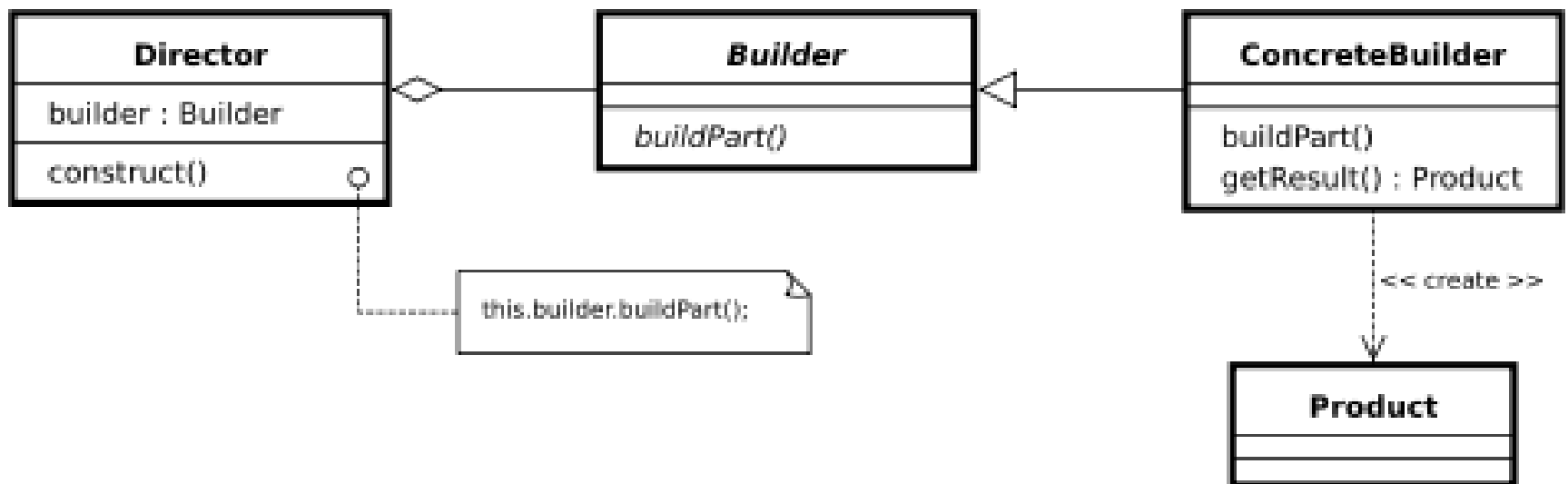
(Builder - Creational)

Builder DP scenario

- Your program has a custom object class that has nearly a hundred private variables – not all of them are necessary in instantiated objects
- To initialize proper variables as/when needed you create overloaded constructors, each with different argument parameters
- Soon, you end up with multiple constructors (and some may be duplicate) which become messy to fix

- This is a common problem especially with generic objects that are instantiated to specific types during object creation – known as the telescoping constructor since addition of a parameter to may lead to exponential number of constructors.
- A better solution is to use the Builder DP to replace ALL constructors with another object, a “builder” that receives building instructions and returns the correct instantiated object

- The main intent of the Builder DP is to separate the construction of a complex object from its representation – i.e. replace the normal constructor



How is it implemented?

- First, let's take a look at a simple class with multiple constructors

```
public class Cake{
    private int eggs;
    private double flour,sugar,butter;
    private String flavour;

    public Cake(){
        eggs = 3; flour = 300; sugar = 100; butter = 1;
    }
    public Cake(int e, double f, double s, double bp){
        eggs = e; flour = f; sugar = s; butter = bp;
    }
    public Cake(int e, double f, double s, double bp,String fr){
        eggs = 3; flour = 300; sugar = 100; butter = 1;
        flavour=fr;
    }
    // public Cake( .....
    // other constructors and methods and stuff
}
```

- To use the Builder DP, the constructors are removed until one remains and is made private – this becomes the “default constructor”.
- This constructor will call the “builder” object

```
public class Cake{  
    private int eggs;  
    private double flour,sugar,butter;  
    private String flavour;  
  
    private Cake(Builder cakemaker){  
        this.eggs = cakemaker.eggs;  
        this.flour = cakemaker.flour;  
        this.sugar = cakemaker.sugar;  
        this.butter = cakemaker.butter;  
        this.flavour = cakemaker.flavour;  
    }  
    // other methods and stuff  
}
```

- Then the builder is created as a static nested class INSIDE the original object
- The builder will have same properties as the original class along with setter functions for each property

```
public static class Builder{  
    private int eggs;  
    private double flour,sugar,butter;  
    private String flavour;  
  
    public Builder eggs(int e){eggs = e; return this;}  
    public Builder flour(double f){flour = f; return this;}  
    public Builder sugar(double s){sugar = s; return this;}  
    public Builder butter(double b){butter = e; return this;}  
    public Builder flavour(String f){flavour = f; return this;}  
  
    public Cake build(){  
        return new Cake(this);  
    }  
}
```



This method will call the constructor of Cake –
eventhough its private, it can do so because this
method belongs to the same class

- The new Cake class with the builder object inside

```
public class Cake{
    private int eggs;
    private double flour,sugar,butter;
    private String flavour;
    private Cake(Builder cakemaker){ // original constructor now private
        this.eggs = cakemaker.eggs;
        this.flour = cakemaker.flour;
        this.sugar = cakemaker.sugar;
        this.butter = cakemaker.butter;
        this.flavour = cakemaker.flavour;
    }
    public static class Builder{ // new static builder object class
        private int eggs;
        private double flour,sugar,butter;
        private String flavour;

        public Builder eggs(int e){eggs = e; return this;}
        public Builder flour(double f){flour = f; return this;}
        public Builder sugar(double s){sugar = s; return this;}
        public Builder butter(double b){butter = b; return this;}
        public Builder flavour(String f){flavour = f; return this;}

        public Cake build(){
            return new Cake(this);
        }
    }
    // other methods and stuff
}
```

- To use the new Cake object, an example would be as follows

```
public class Bakery{
    public static void main(String[] a){
        Cake vanillaCake = new Cake.Builder().sugar(100) .
            butter(200) .eggs(3) .flour(400) .flavour("vanilla") .
            build();

        Cake chocCake = new Cake.Builder().sugar(100) .
            butter(200) .eggs(3) .flour(400) .flavour("choco") .
            build();

        Cake vegeCake = new Cake.Builder().sugar(100) .
            butter(200) .flour(400) .flavour("grass") .build();

        Cake diabetesCake = new Cake.Builder().butter(200) .
            eggs(3) .flour(400) .build();

    }
    // other program code
}
```

Some reasons to use Builder

- Complex object creation – too many parameters, unwieldy.
- Flexibility and readability – method chaining/fluent interface for setting attributes.
- Default values – optional parameters, partially specified configurations.
- Parameter names – for languages without named parameters, like C++ and Java.
- Testability – unit testing objects with specific configurations.
- Maintainability – if object structure changes, easier to modify builder than constructors.

Builder DP summary

Name	Builder
Motivation	The builder pattern is used to create complex objects with constituent parts that must be created in the same order or using a specific algorithm using an external class which controls the construction algorithm
Applicability	Use the Builder pattern when • the algorithm for creating a complex object should be independent of the parts that make up the object and how they're assembled. • the construction process must allow different representations for the object that's constructed
Solution	Typically, there's an abstract Builder class that defines an operation for each component that a director may ask it to create. The operations do nothing by default. A ConcreteBuilder class overrides operations for components it's interested in creating.
Consequences	It lets you vary a product's internal representation It isolates code for construction and representation It gives you finer control over the construction process

For more information on Builder

- <https://www.geeksforgeeks.org/builder-pattern-in-java/>
- Google “Builder Design Pattern” but note that there are some web pages that teaching a very different pattern as the Builder Design Pattern – which is very similar to the Template Method. For the purposes of this class, please use the method taught here, but be aware that some people might use it in that other way.

EXAMPLE

SAMPLE

PROTOTYPE

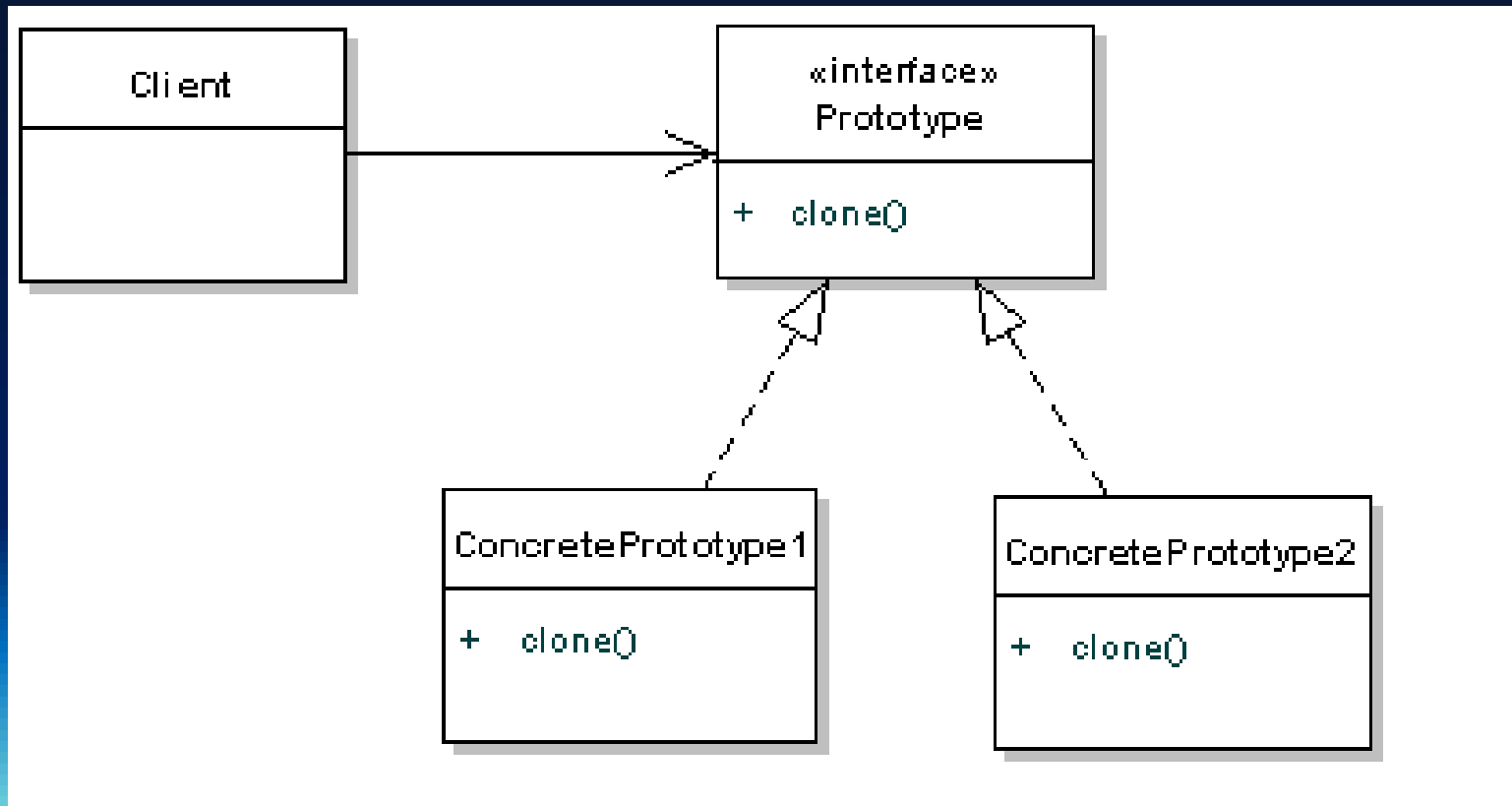
**COMMON DESIGN
PATTERNS**
(Prototype - Creational)

Prototype DP scenario

- Your game developed in Java requires you to create *monsters* to attack the player's character.
- Depending on the character's skill level and game settings, the monsters could number in tens to hundreds.
- To create "new" monsters would slow down the program as it would require constructors to initialize each and every monster

- A better solution is to use the Prototype DP – instead of creating "new" instances of objects, instead it "copies" existing objects with the understanding that later, each copy will have its own set of independent attributes.
- The action of copying an object is faster than new creation – especially if the creation involves heavy cost (processing, data retrieval etc) and the number of objects is unknown before runtime.

- The Prototype DP refers to creating duplicate object while keeping performance in mind
- Involves implementing a prototype interface which has instructions to create a clone of the current object.
- Implementing prototype DP means the objects created are usually similar to existing objects



How is it implemented?

- Using the earlier scenario, imagine the normal method of creating a monster

```
public class Monster{

    String name;
    int level;
    public Monster(String name) {
        this.name = name;
        // some other CPU/storage intensive statement
        // instruction here
        // e.g. retrieve current monster mutation level
        // + monster population health levels etc.
    }
    public String toString() {
        return "This monster is called " + name;
    }
}
```

- Everytime a “new” Monster is created it has to repeat the costly process eventhough it recreates the same Monster

- With prototype DP, an interface is first created ...

```
public interface Prototype {  
    public Prototype doClone();  
}
```

after which the new Monster class is created that implements it

```
public class Monster implements Prototype{  
    String name; int level;  
    public Monster2(String name) {  
        this.name = name;  
        // CPU/storage intensive instruction here  
    }  
    public Monster(String name, int lvl){  
        this.name = name; this level = lvl;  
    }  
    public String toString() {...}  
    @Override  
    public Prototype doClone(){  
        return new Monster(name,level);  
    }  
}
```



this copies data from existing
object instead of retrieving from
database

- So in the game program both Monsters are called and work differently

```
public class BadGame{  
    public static void main(String[] a){  
  
        Monster[] m1 = new Monster[5];  
        for (int i=0;i<5;i++){  
            m1[i] = new Monster("Sulley");  
        }  
  
        Monster[] m2 = new Monster[5];  
        for (int i=0;i<5;i++){  
            m2[i] = m1[0].doClone();  
        }  
    }  
}
```

everytime a "new" Monster is called, a database retrieve operation is performed – thus slowing down performance



this just duplicates the existing Monster object without having to retrieve any additional data



How is it implemented? Again?

- Java has a built-in method which works similar to the Prototype DP – Clone
- Clone works similar to the copy constructor in C++ (please review this if necessary)
- To incorporate cloning into the Monster class (using the example objects from before)

```
public class Monster implements Cloneable {  
    String name; int level;  
    public Monster2(String name) { ..... }  
    public Object clone() throws CloneNotSupportedException{  
        return super.clone();  
    }  
    public String toString() {...}  
}
```

- To “copy” an object, the syntax would then be

```
Monster anotherMonster = m1.clone();
```

Another example

- http://www.tutorialspoint.com/design_pattern/prototype_pattern.htm

Prototype DP summary

Name	Prototype
Motivation	To provide a method to clone/create an object for performance reasons
Applicability	Used when • there is a need to specify classes to instantiate <i>at run time</i> • there is a need to avoid building parallel class hierarchies for factories and products • When instances of a class can have only a few different combinations of state
Solution	Specify the kinds of objects to create using a prototypical instance, and create new object by copying this prototype
Consequences	Hides concrete classes from the client Allows adding and removing products at run-time Allows specifying new object "types" by varying values or structure Can reduce subclassing Dynamic configuration of an application's classes

For more information on Prototype

- <https://refactoring.guru/design-patterns/prototype> points out that another reason you might want to use the prototype design pattern is because it delegates the cloning process to the actual objects that are being cloned. That way, the caller does not need to know the internals of how to clone it. Instead of the caller having to write code to clone it, it delegates the task to the class itself, allowing each class to control exactly how cloning should be handled.
- Google “Prototype Design Pattern”



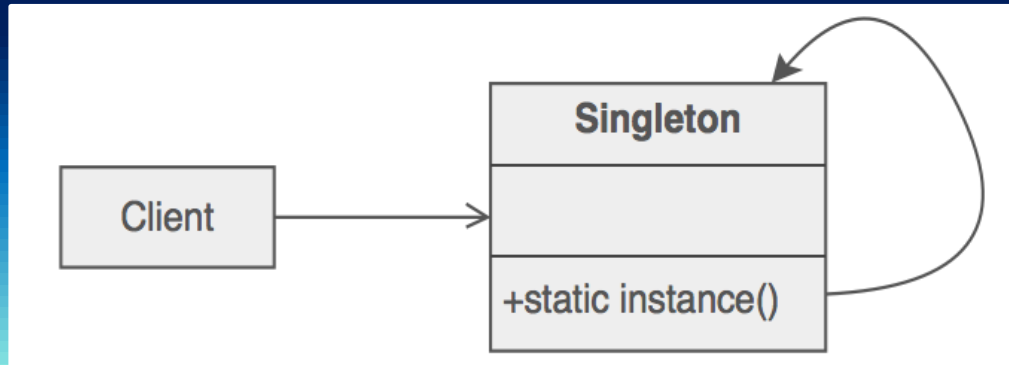
COMMON DESIGN PATTERNS

(Singleton - Creational)

Singleton DP scenario

- You are completing your media player program and to avoid any problems of multiple access to files/stored content, you need it to run as a single instance only – anyways playing multiple media files at once wouldn't make sense anyways

- This is probably the easiest DP called the Singleton which ensures that any particular class has only one instance
- This pattern involves a single class which is responsible to create its own object while making sure that only single object get created.
- This class provides a way to access its only object which can be accessed directly without need to instantiate the object of the class



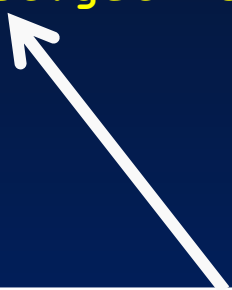
How is it implemented?

- To practise the Singleton DP, a normal class first only needs to have its constructor made as private (so as to disable any instantiations of new objects from the class) and then automatically create a static version of itself with global access.

```
public class SingleObject {  
    // create an single STATIC instance of this class  
    private static SingleObject instance = new SingleObject();  
  
    // make constructor private so cannot be instantiated  
    private SingleObject(){}  
  
    // Method to retrieve the only object instantiated  
    public static SingleObject getInstance(){  
        return instance;  
    }  
  
    public void showMessage(){  
        System.out.println("Hello World!");  
    }  
}
```

- To use the Singleton class in a program ...

```
public class SingletonPatternDemo {  
    public static void main(String[] args) {  
  
        //Compile Time Error: The constructor SingleObject() is  
        //not visible  
        SingleObject object = new SingleObject();  
  
        //Get the only object available  
        SingleObject object = SingleObject.getInstance();  
  
        //show the message  
        object.showMessage();  
    }  
}
```



cannot create “new” but can only
retrieve the static instance of the
object

Why Singleton instead of Class variables?

- Controlled access
- Lazy initialisation
- Thread safety
- Encapsulation and testing

Singleton DP summary

Name	Singleton
Motivation	To ensure a class only has one instance of itself during runtime and to provide a global point of access to it
Applicability	Used when • need exactly one instance of a class and a well-known access point • Need to have the sole instance extensible by subclassing
Solution	Implement class with constructor as private and then create a static instance of itself
Consequences	Need controlled access Reduce name space Permits refinement of operations and representation More flexible than static class operators

For more information on Singleton

- <https://www.geeksforgeeks.org/singleton-design-pattern-in-java/>
- Google “Singleton Design Pattern”

Factory Method - Creational

- The factory pattern is used to replace class constructors, abstracting the process of object generation so that the type of the object instantiated can be determined at run-time.
- <https://www.baeldung.com/java-factory-pattern>
- For more information, Google “Factory Method Design Pattern”, but beware of the fact that many web pages tell you to use switch/case or nested if to do it – this would be *control coupling*, which is bad (see the chapters on OO System Design on coupling.) A better method is to use polymorphism within the factory method.

Abstract Factory - Creational

- The abstract factory pattern is used to provide a client with a set of related or dependent objects. The “family” of objects created by the factory are determined at run-time.
- While **Factory Method** is used when you want to delegate the creation of a single product, the **Abstract Factory** pattern is used when you need to create **families of related products**.
- <https://www.baeldung.com/java-abstract-factory-pattern>
- For more information, Google “Abstract Factory Design Pattern”

Choosing design patterns

- Patterns are not the simple answer to program design.
- They do not and cannot replace traditional object-oriented analysis techniques like CRC cards or use-case modelling.
 - To use an architectural analogy, domain analysis lets you determine that a house needs 200 amps of electricity. The DP just determines how the wiring will be installed.
- Patterns only help you think about the problems you may encounter while designing a system.
- Therefore, a generic, pattern-based solution is often more robust than a solution designed by one individual to solve a specific problem.
- The first thing you should decide is whether the problem is fundamentally creational, structural, or behavioural.
- Some problems, of course, have aspects of two or even three, and may require you to mix and match patterns.

Heuristics for identifying DP

- Certain key phrases can often be used to identify candidate/ potential design patterns for implementation
- For example,

"Must comply with existing interfaces" "Must reuse existing legacy component"	→	Adapter
"Must support future protocols or DBs"	→	Bridge
"Must support aggregate structures" "Must allow for hierarchies of variable depth"	→	Composite
"Must support new views or visualizations"	→	MVC

Patterns and changes they anticipate

Design Pattern	Anticipated Change
Bridge	New Vendor, new technology, new implementation: decoupling the interface from its implementation. Same as adapter, but developer is not constrained by an existing component
Adapter	New Vendor, new technology, new implementation: encapsulating a piece of legacy code that was not designed to work with the system
Composite	New Complexity, new functions, new activities: encapsulates hierarchies by providing a common superclass for aggregate and leaf nodes. New leafs can be added without modifying existing code.
Model-View-Controller (MVC)	New Views. Decouples data models from their actual representation (view). If data is changed, all views are updated consistently. New Views can be added independently from changing any view or the model

Summary

- Design patterns are partial solutions to common problems such as separating an interface from a number of alternate implementations, wrapping around a set of legacy classes and protecting a caller from changes associated with specific platforms.
- DPs assist in creating better software design with methods that have been tried and tested